《数据结构基础》(FDS) 专项练习题

Part 1: Priority Queues & Disjoint Sets

Based on FDS Midterm Reviews

说明：本练习集严格按照往年 FDS 期中真题的题型（判断、选择、程序填空、函数题）编写，本卷包含讲义中【第四章：优先队列 (Heaps)】与【第五章：并查集 (Disjoint Sets)】的独立专项训练。

Chapter 4: Priority Queues (Heaps) 专项练习

一、判断题 (True or False)

1-1. A sequence of N insertions into an initially empty max-heap takes $O(N)$ time in the worst case. (T / F)

1-2. In a min-heap, the elements on any path from the root to a leaf are in sorted (non-decreasing) order. (T / F)

1-3. The time complexity to find the maximum element in a min-heap of size N is $O(1)$. (T / F)

1-4. The `BuildHeap` operation (linear time algorithm) takes $O(N)$ time to arrange N elements into a heap. (T / F)

二、选择题 (Multiple Choice)

2-1. Given an initially empty min-heap, we insert the elements {10, 12, 1, 14, 6, 5, 8, 15, 3, 9, 7} one by one. Which of the following is the array representation of the resulting heap?

A. 1, 3, 5, 10, 6, 12, 8, 15, 14, 9, 7

B. 1, 3, 5, 10, 7, 12, 8, 15, 14, 9, 6

C. 1, 3, 5, 14, 6, 12, 8, 15, 10, 9, 7

D. 1, 6, 5, 10, 3, 12, 8, 15, 14, 9, 7

2-2. Which of the following sequences could be the array representation of a max-heap?

A. 10, 5, 8, 2, 6, 7, 1

B. 10, 8, 5, 6, 2, 1, 7

C. 10, 8, 5, 2, 7, 6, 1

D. 10, 5, 8, 6, 2, 1, 7

2-3. In a complete binary tree (which represents a heap) with 1102 nodes, there must be _____ leaf nodes.

A. 79

B. 551

C. 1063

D. cannot be determined

三、程序填空题 (Fill in the blanks)

3-1. The following function is to lower the value of the integer key at position `P` by a positive amount `D` in a min-heap `H` (This operation is called `DecreaseKey`). Please complete the code.

```
void DecreaseKey( int P, int D, PriorityQueue H ) {
    int i, key;
    key = H->Elements[P] - D;

    /* Percolate up */
    for ( i = P; ________(1)________; i /= 2 ) {
        H->Elements[i] = H->Elements[i / 2];
    }
    ________(2)________;
}
```

四、函数题 (Programming)

4-1. Delete Minimum from Min-Heap

Write the function to delete the minimum element from a Min-Heap and return its value. The heap is guaranteed to be non-empty.

```
ElementType DeleteMin( PriorityQueue H );
```

Note: `H->Elements[0]` is a sentinel value which is smaller than any possible key. `H->Size` records the current number of elements.

Chapter 5: The Disjoint Set ADT 专项练习

一、判断题 (True or False)

1-1. In Union/Find algorithm, if Unions are done by size, the depth of any node must be no more than $O(\log N)$. (T / F)

1-2. Path compression is fully compatible with Union-by-Height without requiring any modification to how heights are tracked or estimated. (T / F)

1-3. An array representation of a disjoint set initialized entirely with -1 indicates that each element is a disjoint root of a tree with size 1. (T / F)

二、选择题 (Multiple Choice)

2-1. Let $T(M, N)$ be the maximum time required to process an intermixed sequence of $M \geq N$ `Find`s and $N - 1$ `Union`s. With path compression and union-by-rank, the time complexity is bounded by:

A. $O(M \log N)$

B. $O(M + N)$

C. $O(M\alpha(M, N))$

D. $O(N \log M)$

2-2. We use an array `S` to represent a Disjoint Set. If we apply **Union-by-Size**, what is the value stored in the root's array index after unioning a tree of size 3 and a tree of size 5?

A. 8

B. -8

C. -2

D. -15

三、程序填空题 (Fill in the blanks)

3-1. The following function implements the `Find` operation with **Path Compression**. Please complete the blanks.

```
SetType Find( ElementType X, DisjSet S ) {
    if ( S[X] <= 0 )
        return ________(1)________;
    else
        return ________(2)________;
}
```

四、函数题 (Programming)

4-1. Set Union by Height

Write the function to perform a union of two disjoint sets using the **Union-by-Height** strategy. `Root1` and `Root2` are guaranteed to be the roots of two different trees. In this array representation, the root contains the negative of the tree height.

```
void SetUnion( DisjSet S, SetType Root1, SetType Root2 );
```

参考答案与解析 (Solutions for Ch4 & Ch5)

Chapter 4: Priority Queues (Heaps)

一、判断题

1. **F**. N successive insertions take $O(N \log N)$ time in the worst case (each insertion is $O(\log N)$). Only the `BuildHeap` operation takes $O(N)$.
2. **T**. By definition, a parent is always smaller than or equal to its children in a min-heap, so the path is non-decreasing.
3. **F**. To find the maximum in a min-heap, we must inspect all the leaf nodes (around $N/2$ nodes), which takes $O(N)$ time.
4. **T**. Standard Tarjan's analysis shows `BuildHeap` works in $O(N)$.

二、选择题

1. **A**. (Simulating insertions and percolating up yields option A).
2. **A**. In a max-heap, parent $\geq$ children. Option A represents tree: 10(root), left 5, right 8. Children of 5 are 2, 6. Children of 8 are 7, 1. All paths satisfy max-heap property.
3. **B**. In a complete binary tree of N nodes, the number of leaves is $\lceil N/2 \rceil$. So $\lceil 1102/2 \rceil = 551$.

三、程序填空题

1. `H->Elements[i / 2] > key` (上滤操作：如果父节点的值大于减小后的 key，则将父节点下移)
2. `H->Elements[i] = key` (找到合适的位置后，将减小后的 key 放入)

四、函数题

```
ElementType DeleteMin( PriorityQueue H ) {
    int i, Child;
    ElementType MinElement, LastElement;

    if ( H->Size == 0 ) return H->Elements[0]; // Error handling

    MinElement = H->Elements[1];
    LastElement = H->Elements[H->Size--];

    /* Percolate Down */
    for ( i = 1; i * 2 <= H->Size; i = Child ) {
        Child = i * 2;
```

```
        if ( Child != H->Size && H->Elements[Child + 1] < H->Elements[Child
            ] )
            Child++;
        if ( LastElement > H->Elements[Child] )
            H->Elements[i] = H->Elements[Child];
        else
            break;
    }
    H->Elements[i] = LastElement;
    return MinElement;
}
```

Chapter 5: The Disjoint Set ADT

一、判断题

1. **T**. Union-by-size guarantees that the height (depth) of any tree with N nodes is at most $\lfloor \log_2 N \rfloor$.

2. **F**. Path compression changes the exact height of the trees, making the stored "height" an overestimation (often called "rank"). Strict Union-by-Height is not perfectly maintained.

3. **T**. -1 represents the root of a set whose size/height is 1.

二、选择题

1. **C**. Tarjan's Theorem bounds the complexity to $O(M\alpha(M, N))$, where α is the inverse Ackermann function.

2. **B**. In Union-by-Size, roots store the negative of their sizes. Total size is $3 + 5 = 8$, so it stores -8.

三、程序填空题

1. `X` (如果 S[X] <= 0，说明 X 本身就是根节点，直接返回)

2. `S[X] = Find( S[X], S )` (递归寻找根节点，并把沿途的所有节点直接挂到根节点上实现路径压缩)

四、函数题

```
void SetUnion( DisjSet S, SetType Root1, SetType Root2 ) {
    /* Height is stored as a negative number.
       More negative means deeper tree. */
    if ( S[Root2] < S[Root1] ) {
```

```
5           /* Root2 is deeper */
6           S[Root1] = Root2; /* Make Root2 new root */
7       } else {
8           if ( S[Root1] == S[Root2] ) {
9               /* Same height, so update height of Root1 */
10              S[Root1]--;
11          }
12          S[Root2] = Root1; /* Make Root1 new root */
13      }
14  }
```